Cidades ESG Inteligentes

Documentação técnica - DevOps, CI/CD e Containerização

Integrante: Vitor Pavan

Stack: C# / .NET 8 / ASP.NET Core / SQLite / Docker / GitHub Actions

Atividade Avaliativa - Navegando pelo mundo DevOps

1. Descrição do projeto

O projeto Cidades ESG Inteligentes é uma API REST para registro e consulta de indicadores de sustentabilidade urbana, como consumo de energia, reciclagem e áreas verdes. A solução foi preparada para demonstrar um ciclo DevOps completo.

A API oferece CRUD, Swagger para documentação interativa e o endpoint /health para validação operacional.

2. Arquitetura

A aplicação utiliza ASP.NET Core Web API em .NET 8 e Entity Framework Core com SQLite. O arquivo do banco é persistido em volume Docker separado para cada ambiente. Docker empacota a aplicação e o Docker Compose cria staging e produção.

Os ambientes usam a mesma base de código e imagem, alterando ASPNETCORE_ENVIRONMENT, porta e volume.

3. Pipeline CI/CD

O pipeline está em .github/workflows/ci-cd.yml e usa GitHub Actions. O job inicial executa checkout, .NET 8, restore, build, testes automatizados e build da imagem Docker.

Depois, staging sobe o container e executa smoke test em /health. Produção depende do sucesso de staging e executa a mesma validação. Os deploys do workflow são efêmeros no runner do GitHub, adequados à demonstração acadêmica; em produção real, o destino pode ser Azure, Kubernetes ou servidor persistente.

4. Docker e orquestração

O Dockerfile usa multi-stage build: o SDK .NET 8 compila/publica e a imagem final usa ASP.NET Runtime.

O Docker Compose define api-staging e api-production, portas distintas, variáveis de ambiente, volumes persistentes SQLite e a rede bridge esg-network.

5. Testes automatizados

O projeto contém testes xUnit com WebApplicationFactory. São validados /health e /api/indicadores. O pipeline executa dotnet test antes dos deploys e realiza smoke tests após subir cada ambiente.

6. Comandos principais

Subir: docker compose up --build -d

Listar: docker compose ps

Encerrar: docker compose down

Testar: dotnet test CidadesESG.sln -c Release

Staging: http://localhost:8081/health

Produção: http://localhost:8080/health

Swagger: http://localhost:8080/swagger

7. Evidências obrigatórias

Antes da entrega, inserir prints reais do GitHub Actions mostrando Build e testes, Deploy - Staging e Deploy - Production concluídos; docker compose ps; /health em staging e produção; e Swagger.

Importante: os prints não foram fabricados. Eles precisam ser capturados após a execução na conta/máquina do aluno.

Evidência	Status
GitHub Actions - Build e testes	PENDENTE: print real
Deploy - Staging	PENDENTE: print real
Deploy - Production	PENDENTE: print real
docker compose ps	PENDENTE: print real
Health staging e produção	PENDENTE: prints reais
Swagger / API	PENDENTE: print real

8. Desafios e soluções

Padronização: Docker e variáveis de ambiente.

Persistência: SQLite em volumes Docker separados.

Qualidade: build/testes bloqueiam staging; staging bloqueia produção.

Validação: /health e smoke tests automatizados.

9. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	SIM
Dockerfile funcional	SIM*
docker-compose.yml / orquestração	SIM*
Pipeline com build, teste e deploy	SIM*
README.md com instruções	SIM
Documentação técnica em PDF/PPT	SIM
Deploy configurado para staging e produção	SIM
Prints reais de execução anexados	PENDENTE

* Arquivos revisados estaticamente. A execução real depende de Docker/.NET/GitHub, indisponíveis no ambiente de geração deste pacote.

Antes do upload: execute o workflow e substitua os placeholders por evidências reais.